11
Fast Euclidean Algorithm

The main result of this chapter is a fast algorithm for the quotients in the Euclidean Algorithm for univariate polynomials over a field, using $O(\mathsf{M}(n)\log n)$ field operations for inputs of degree at most n. One can also compute a single remainder r_i together with the corresponding s_i and t_i, at the same cost, but this is not possible for all remainders together. The final section shows how one can also calculate subresultants in softly linear time.

In order to make the description and the proofs simpler, the fast Euclidean Algorithm that we present in this section will not make the remainders in intermediate computations monic, but only at the end, if desired. For that reason, it is most suitable for coefficient fields with a fixed size representation for the elements, such as finite fields. For other coefficient domains, it is best to use a combination of the modular algorithms described in Chapter 6 with the algorithm in this chapter.

11.1. A fast Euclidean Algorithm for polynomials

Let F be a field, $r_0, r_1 \in F[x]\setminus\{0\}$ with $\deg r_0 \geq \deg r_1$, $s_0 = t_1 = 1$, $s_1 = t_0 = 0$, and

$$\begin{array}{rclrclrcl}
r_2 &=& r_0 - q_1 r_1, & s_2 &=& s_0 - q_1 s_1, & t_2 &=& t_0 - q_1 t_1, \\
&\vdots& & &\vdots& & &\vdots& \\
r_{i+1} &=& r_{i-1} - q_i r_i, & s_{i+1} &=& s_{i-1} - q_i s_i, & t_{i+1} &=& t_{i-1} - q_i t_i, \\
&\vdots& & &\vdots& & &\vdots& \\
0 &=& r_{\ell-1} - q_\ell r_\ell, & s_{\ell+1} &=& s_{\ell-1} - q_\ell s_\ell, & t_{\ell+1} &=& t_{\ell-1} - q_\ell t_\ell
\end{array} \tag{1}$$

be the results of the traditional Extended Euclidean Algorithm for r_0, r_1, where $\deg r_{i+1} < \deg r_i$ for $1 \leq i < \ell$, as introduced in Section 3.2. We will assume that $r_{\ell+1} = 0$ and $\deg r_{\ell+1} = -\infty$. Then Lemma 3.8 (i) says that

$$\begin{pmatrix} r_i \\ r_{i+1} \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & -q_i \end{pmatrix} \begin{pmatrix} r_{i-1} \\ r_i \end{pmatrix} = Q_i \begin{pmatrix} r_{i-1} \\ r_i \end{pmatrix} = Q_i \cdots Q_1 \begin{pmatrix} r_0 \\ r_1 \end{pmatrix},$$

https://doi.org/10.1017/CBO9781139856065.014 Published online by Cambridge University Press

where

$$Q_i = \begin{pmatrix} 0 & 1 \\ 1 & -q_i \end{pmatrix} \in F[x]^{2\times 2} \text{ and } R_i = Q_i \cdots Q_1 = \begin{pmatrix} s_i & t_i \\ s_{i+1} & t_{i+1} \end{pmatrix}. \tag{2}$$

Let $n_i = \deg r_i$ for $0 \le i \le \ell$ and $m_i = \deg q_i = n_{i-1} - n_i$ for $1 \le i \le \ell$. The sequence $(n_0, n_1, \ldots, n_\ell)$ is the **degree sequence** in the Extended Euclidean Algorithm for r_0, r_1. If $F = \mathbb{F}_q$ is a finite field with q elements and $r_0, r_1 \in \mathbb{F}_q[x]$ with $\deg r_0 = n_0 > \deg r_1 = n_1$ are uniform random polynomials, then

$$\text{prob}(\deg r_2 < n_1 - 1) = \frac{1}{q}$$

(Exercise 4.18), which is rather small for large q. So typically one can expect that the degree of each quotient is 1, or equivalently, that $n_{i+1} = n_i - 1$ for $1 \le i < \ell$. In that case, we call the degree sequence $(n_0, \ldots, n_\ell)$ **normal**.

The basic idea leading to a fast gcd algorithm is that the first quotients q_i only depend on the highest coefficients of r_0 and r_1. To express this idea formally, we introduce some notation.

Let $f = f_n x^n + f_{n-1}x^{n-1} + \cdots + f_0 \in F[x]$ with leading coefficient $f_n \neq 0$, and $k \in \mathbb{Z}$. Then we define the truncated polynomial

$$f \upharpoonright k = f \text{ quo } x^{n-k} = f_n x^k + f_{n-1}x^{k-1} + \cdots + f_{n-k},$$

where we set $f_i = 0$ for $i < 0$. So for $k \ge 0$, $f \upharpoonright k$ is a polynomial of degree k whose coefficients are the $k+1$ highest coefficients of f, and $f \upharpoonright k = 0$ if $k < 0$. We also define $f \upharpoonright -\infty = 0$, and $0 \upharpoonright k = 0$ for all $k \in \mathbb{Z} \cup \{-\infty\}$. For all $i \ge 0$ we have that $(fx^i) \upharpoonright k = f \upharpoonright k$.

Now let $f, g, f^*, g^* \in F[x]$ with f, f^* both nonzero, $\deg f \ge \deg g$ and $\deg f^* \ge \deg g^*$, and $k \in \mathbb{Z}$. Then **(f, g) and (f^*, g^*) coincide up to k** if

$$f \upharpoonright k = f^* \upharpoonright k,$$
$$g \upharpoonright (k - (\deg f - \deg g)) = g^* \upharpoonright (k - (\deg f^* - \deg g^*)).$$

This defines an equivalence relation on $F[x] \setminus \{0\} \times F[x]$ (Exercise 11.1). If (f, g) and (f^*, g^*) coincide up to k and $k \ge \deg f - \deg g$, then $\deg f - \deg g = \deg f^* - \deg g^*$.

We consider one division step in the Euclidean Algorithm.

LEMMA 11.1. *Let $k \in \mathbb{N}$, (f, g) and (f^*, g^*) in $(F[x]\setminus\{0\})^2$ coincide up to $2k$, and $k \ge \deg f - \deg g \ge 0$. Define $q, r, q^*, r^* \in F[x]$ by division with remainder:*

$$\begin{aligned} f &= qg + r, & \deg r &< \deg g, \\ f^* &= q^* g^* + r^*, & \deg r^* &< \deg g^*. \end{aligned}$$

Then $q = q^*$, and either (g, r) and (g^*, r^*) coincide up to $2(k - \deg q)$, or $r = 0$, or $k - \deg q < \deg g - \deg r$.

https://doi.org/10.1017/CBO9781139856065.014 Published online by Cambridge University Press

PROOF. By multiplying (f,g) and (f^*,g^*) with appropriate powers of x if necessary, we may assume that $\deg f = \deg f^* > 2k$ holds. Then $\deg g = \deg g^*$, $k \geq \deg q = \deg f - \deg g = \deg f^* - \deg g^* = \deg q^*$,

$$\begin{aligned} \deg(f-f^*) &< \deg f - 2k \leq \deg g - k, \\ \deg(g-g^*) &< \deg g - (2k - (\deg f - \deg g)) = \deg f - 2k \\ &\leq \deg g - k \leq \deg g - \deg q, \\ \deg(r-r^*) &\leq \max\{\deg r, \deg r^*\} < \deg g, \end{aligned} \tag{3}$$

and

$$f - f^* = q(g-g^*) + (q-q^*)g^* + (r-r^*). \tag{4}$$

The polynomials $f-f^*$, $q(g-g^*)$ and $r-r^*$ all have degree less than $\deg g$ by (3), hence also $\deg((q-q^*)g^*) < \deg g = \deg g^*$, which implies that $q = q^*$.

Now we assume that $r \neq 0$ and $k - \deg q \geq \deg g - \deg r$. We have to show that

$$\begin{aligned} g \restriction (2(k-\deg q)) &= g^* \restriction (2(k-\deg q)), \\ r \restriction (2(k-\deg q) - (\deg g - \deg r)) &= r^* \restriction (2(k-\deg q) - (\deg g^* - \deg r^*)). \end{aligned}$$

The first assertion follows from the coincidence up to $2k$ of (f,g) and (f^*,g^*). Furthermore we have

$$\begin{aligned} \deg(r-r^*) &\leq \max\{\deg(f-f^*), \deg q + \deg(g-g^*)\} \\ &< \deg q + \deg f - 2k = \deg g - 2(k - \deg q) \\ &= \deg r - (2(k-\deg q) - (\deg g - \deg r)), \end{aligned} \tag{5}$$

by (4) and (3), and by the above assumption

$$\deg r \geq \deg q + \deg g - k \geq \deg q + \deg f - 2k > \deg(r-r^*),$$

so that $\deg r = \deg r^*$. Now the second assertion follows from the second inequality in (5). □

EXAMPLE 11.2. Let

$$\begin{aligned} f &= x^8 + 5x^7 + 3x^6 + 5x^4 + 4x^3 + 5x^2 + 2x + 2, \\ g &= x^7 + 4x^6 + 4x^5 + 2x^4 + x^3 + 4x^2 + x + 3 \end{aligned}$$

be polynomials over $\mathbb{F}_7$, and

$$f^* = f \restriction 4 = x^4 + 5x^3 + 3x^2 + 5, \quad g^* = g \restriction 3 = x^3 + 4x^2 + 4x + 2.$$

Then (f,g) and $(f \restriction 4, g \restriction 3)$ coincide up to $2 \cdot 2 = 4$, so that $k = 2$. In the proof of Lemma 11.1, we multiply the second pair with x^4 and obtain

$$f^* = x^8 + 5x^7 + 3x^6 + 5x^4, \quad g^* = x^7 + 4x^6 + 4x^5 + 2x^4.$$

https://doi.org/10.1017/CBO9781139856065.014 Published online by Cambridge University Press

Then

$$\begin{aligned} q &= x+1, & r &= 2x^6+x^5+2x^4+6x^3+5x+6,\\ q^* &= x+1, & r^* &= 2x^6+x^5+3x^4. \end{aligned}$$

We see that $q = q^*$ and $r \upharpoonright 1 = r^* \upharpoonright 1$, and since $g \upharpoonright 2 = g^* \upharpoonright 2$, we have that (g,r) and (g^*,r^*) coincide up to $2 = 2(k - \deg q)$. Now $r^*/x^4 = (f \upharpoonright 4) \text{ rem } (g \upharpoonright 3)$, and we find that also (g,r) and $(g \upharpoonright 3, (f \upharpoonright 4) \text{ rem } (g \upharpoonright 3))$ coincide up to $2(k - \deg q)$, as stated in the lemma. $\diamond$

Lemma 11.1 gives only sufficient conditions for the quotients to be equal. Often less information is necessary; in the above example, the constant coefficient of f^* may be altered without changing the quotient.

Next we consider the Euclidean Algorithm for two pairs r_0, r_1 and r_0^*, r_1^* of polynomials with $\deg r_0 > \deg r_1$ and $\deg r_0^* > \deg r_1^*$:

$$\begin{aligned} r_0 &= q_1 r_1 + r_2, & r_0^* &= q_1^* r_1^* + r_2^*,\\ &\vdots & &\vdots\\ r_{i-1} &= q_i r_i + r_{i+1}, & r_{i-1}^* &= q_i^* r_i^* + r_{i+1}^*,\\ &\vdots & &\vdots\\ &\vdots & r_{\ell^*-1}^* &= q_{\ell^*}^* r_{\ell^*}^*,\\ r_{\ell-1} &= q_\ell r_\ell \end{aligned}$$

of length ℓ and ℓ^*, respectively, and let $m_i = \deg q_i$ for $1 \le i \le \ell$ and $m_i^* = \deg q_i^*$ for $1 \le i \le \ell^*$. As usual, we let $n_i = \deg r_i = n_0 - m_1 - \cdots - m_i$ for $0 \le i \le \ell$ and $n_{\ell+1} = -\infty$, and we define n_i^* accordingly, for $0 \le i \le \ell^* + 1$.

Moreover, we define for any $k \in \mathbb{N}$ the number $\eta(k) \in \mathbb{N}$ by

$$\eta(k) = \max\{0 \le j \le \ell : \sum_{1 \le i \le j} m_i \le k\},$$

so that

$$n_0 - n_{\eta(k)} = \sum_{1 \le i \le \eta(k)} m_i \le k < \sum_{1 \le i \le \eta(k)+1} m_i = n_0 - n_{\eta(k)+1}, \tag{6}$$

where the second inequality only holds if $\eta(k) < \ell$, and $\eta(k)$ is uniquely determined by (6). In other words, the number $n_0 - k$ is sandwiched between the two consecutive remainder degrees $\deg r_{\eta(k)}$ and $\deg r_{\eta(k)+1}$ in the Euclidean Algorithm. In particular, $\eta(k) \le k$ since $m_i \ge 1$ for $1 \le i \le \ell$. We define $\eta^*(k)$ analogously. The following lemma says, in a precise way, that the first results in the Euclidean Algorithm only depend on the top part of the inputs.

LEMMA 11.3. *Let $k \in \mathbb{N}$, $h = \eta(k)$, and $h^* = \eta^*(k)$. If (r_0, r_1) and (r_0^*, r_1^*) coincide up to $2k$, then $h = h^*$ and $q_i = q_i^*$ for $1 \le i \le h$.*

https://doi.org/10.1017/CBO9781139856065.014 Published online by Cambridge University Press

PROOF. We show by induction on j that the following holds for $0 \le j \le h$:

$$j \le h^* \text{ and } q_i = q_i^* \text{ for } 1 \le i \le j, \text{ and}$$
$$\text{either } j = h \text{ or } (r_j, r_{j+1}) \text{ and } (r_j^*, r_{j+1}^*) \text{ coincide up to } 2(k - n_0 + n_j).$$

The assertion of the lemma follows from this and the symmetric statement.

There is nothing to prove for $j = 0$. Let us assume the induction hypothesis holds for $j - 1 \ge 0$. Then $j - 1 < h$ implies $r_{j-1} \ne 0$, $k \ge n_0 - n_j \ge n_{j-1} - n_j = n_{j-1}^* - n_j^*$, and hence also $r_j^* \ne 0$. Lemma 11.1, with k replaced by $k - n_0 + n_{j-1}$, implies that $q_j = q_j^*$, and either (r_j, r_{j+1}) and (r_j^*, r_{j+1}^*) coincide up to $2(k - n_0 + n_j)$, or $r_{j+1} = 0$, or $k - n_0 + n_j < n_j - n_{j+1}$. In the second case, we have $j + 1 > l$, $j \ge 1$, and hence $j = h = 1$. In the third case we have $h = \eta(k) = j$, by the definition of η. Finally, the assertion $j \le h^*$ is true because

$$\sum_{1 \le i \le j} \deg q_i^* = \sum_{1 \le i \le j} \deg q_i \le \sum_{1 \le i \le h} \deg q_i \le k,$$

and hence $j \le \eta^*(k) = h^*$. This concludes the induction step. □

We use this lemma to construct a divide-and-conquer algorithm for a single row in the traditional Extended Euclidean Algorithm, say the last one. One has to be careful about dividing the problem into two subproblems of about the same size. An appealing idea is to first compute the first $\ell/2$ of the quotients q_i and then the second half, where ℓ is the number of division steps as usual, but this is not the best way because in general the quotients may have very different degrees. It is better to divide the sequence of the quotients into two parts such that the sum of the degrees in the first part is about the same as in the second part. In the case of a normal degree sequence, this will of course yield the same result as the first idea.

The following algorithm uses this strategy. For simplicity, it does not return the quotients, but instead returns the product of the corresponding Q-matrices. Moreover, in order to simplify the presentation and the analysis, we assume for the moment that the degree sequence is normal.

ALGORITHM 11.4 Half gcd for normal degree sequence.
Input: $r_0, r_1 \in F[x]$, $n = \deg r_0 \ge 0$, $\deg r_1 = n - 1$ if $n > 0$ and $r_1 = 0$ if $n = 0$, and $k \in \mathbb{N}$ with $0 \le k \le n$.
Output: $R_k \in F[x]^{2\times 2}$ as in (2).

1. **if** $k = 0$ **then return** $\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$
2. $d \longleftarrow \lceil k/2 \rceil$
3. **call** the algorithm recursively with input $r_0 \restriction (2d-2)$, $r_1 \restriction (2d-3)$ and $d-1$ (instead of k), returning $R = Q_{d-1} \cdots Q_1$

https://doi.org/10.1017/CBO9781139856065.014 Published online by Cambridge University Press

4. $\begin{pmatrix} r_{d-1} \\ r_d \end{pmatrix} \longleftarrow R \begin{pmatrix} r_0 \\ r_1 \end{pmatrix}$

5. $q_d \longleftarrow r_{d-1} \text{ quo } r_d, \quad r_{d+1} \longleftarrow r_{d-1} \text{ rem } r_d, \quad Q_d \longleftarrow \begin{pmatrix} 0 & 1 \\ 1 & -q_d \end{pmatrix}$

6. $d^* \longleftarrow \lfloor k/2 \rfloor$

7. **call** the algorithm recursively with input $r_d \restriction 2d^*$, $r_{d+1} \restriction (2d^* - 1)$ and d^* (instead of k), returning $S = Q_k \cdots Q_{d+1}$

8. **return** SQ_dR

EXAMPLE 11.2 (continued). Let $r_0 = f$ and $r_1 = g$, where f and g in $\mathbb{F}_7[x]$ are as in Example 11.2, and $k = 5$. The quotients in the Euclidean Algorithm for r_0 and r_1 are

$$\begin{array}{llll} q_1 = x+1, & q_2 = 4x, & q_3 = 3x+6, & q_4 = 2x+3, \\ q_5 = 4x, & q_6 = 2x, & q_7 = 5x, & q_8 = 4x+2, \end{array}$$

of degrees $m_i = 1$ for $1 \le i \le 8$, and the length is $\ell = 8$. The half gcd algorithm proceeds as follows.

2. $d = \lceil 5/2 \rceil = 3$.

3. The recursive call with $r_0 \restriction (2 \cdot 3 - 2) = x^4 + 5x^3 + 3x^2 + 5$, $r_1 \restriction (2 \cdot 3 - 3) = x^3 + 4x^2 + 4x + 2$ and $d - 1 = 2$ yields

$$\begin{aligned} R = Q_2 Q_1 &= \begin{pmatrix} 0 & 1 \\ 1 & 3x \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 6x+6 \end{pmatrix} \\ &= \begin{pmatrix} 1 & 6x+6 \\ 3x & 4x^2+4x+1 \end{pmatrix} = \begin{pmatrix} s_2 & t_2 \\ s_3 & t_3 \end{pmatrix}. \end{aligned}$$

4. $\begin{pmatrix} r_2 \\ r_3 \end{pmatrix} = R \begin{pmatrix} r_0 \\ r_1 \end{pmatrix} = \begin{pmatrix} 2x^6 + x^5 + 2x^4 + 6x^3 + 5x + 6 \\ 3x^5 + 6x^4 + x^3 + 5x^2 + 5x + 3 \end{pmatrix}$.

5. Here q_3, $r_4 = 5x^4 + 6x^3 + 4x^2 + x + 2$, and Q_3 are computed.

6. $d^* \longleftarrow 2$.

7. The recursive call with $r_3 \restriction (2 \cdot 2) = 3x^4 + 6x^3 + x^2 + 5x + 5$, $r_4 \restriction (2 \cdot 2 - 1) = 5x^3 + 6x^2 + 4x + 1$ and d^* returns

$$S = Q_5 Q_4 = \begin{pmatrix} 0 & 1 \\ 1 & 3x \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 5x+4 \end{pmatrix} = \begin{pmatrix} 1 & 5x+4 \\ 3x & x^2+5x+1 \end{pmatrix}.$$

https://doi.org/10.1017/CBO9781139856065.014 Published online by Cambridge University Press

8. The matrix

$$SQ_3R = \begin{pmatrix} 1 & 5x+4 \\ 3x & x^2+5x+1 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 4x+1 \end{pmatrix} \begin{pmatrix} 1 & 6x+6 \\ 3x & 4x^2+4x+1 \end{pmatrix}$$
$$= \begin{pmatrix} 4x^3+6x+4 & 3x^3+3x^3+4x+1 \\ 5x^4+2x^2+x+1 & 2x^5+2x^4+2x^3+4x^2+3x \end{pmatrix} = \begin{pmatrix} s_5 & t_5 \\ s_6 & t_6 \end{pmatrix}$$

is returned. ◇

In the following correctness proof, we denote by log the binary logarithm.

THEOREM 11.5.

In the case of a normal degree sequence, Algorithm 11.4 works correctly. If $k \geq n/2$, it uses no more than $(12\mathsf{M}(k)+O(k))\log k$ additions and multiplications plus k inversions, in total $O(\mathsf{M}(k)\log k)$ operations in F.

PROOF. Since the empty product is defined to be the multiplicative neutral element, in this case the identity matrix, the statements of the theorem are satisfied if $k = 0$. Otherwise, we have $n > 0$, and we see by induction on k and Lemma 11.3 that the results of the recursive call in step 3 are correct. Since the degree sequence is normal, we have $\deg r_d = n - d \geq 0$ and therefore $r_d \neq 0$ in step 5. Again by induction and Lemma 11.3, the results of the recursive call in step 7 are correct, and hence also the final result in step 8.

We may arrange things in such a way that the only inversions in F take place in step 5, and that $\mathrm{lc}(r_j)^{-1}$ is computed only once. During the recursive process, step 5 is executed k times.

Let $T(k)$ denote the number of additions and multiplications that the algorithm uses on input k. Since $d - 1 = \lceil k/2 \rceil - 1 \leq \lfloor k/2 \rfloor$, assuming monotonicity of T, steps 3 and 7 take at most $T(\lfloor k/2 \rfloor)$ operations each for solving a subproblem of the same kind. We now analyze the cost for the polynomial multiplications, divisions, and additions in steps 4, 5 and 8. We note that, by our assumption, $n_i = \deg r_i = n - i$ for $0 \leq i \leq n$.

In step 4, the entries of R are $s_{d-1}, t_{d-1}, s_d, t_d$, by Lemma 3.8 (ii), and their degrees are $n_1 - n_{d-2}, n_0 - n_{d-2}, n_1 - n_{d-1}, n_0 - n_{d-1}$, by Lemma 3.10. All four degrees are at most $d - 1 < k/2$. We have four multiplications of polynomials of degree at most $k/2$ by polynomials of degree at most $n \leq 2k$, plus some additions. Dividing the larger polynomials into blocks of degree at most $k/2$ (Exercise 8.35), the cost for step 4 is $16\mathsf{M}(k/2) + O(k)$, or $8\mathsf{M}(k) + O(k)$, by the superlinearity properties (9) in Section 8.3.

Since $n_{d-1} = n_d + 1$, the cost for step 5 is $O(k)$. In step 8, we first compute $Q_d R$, or equivalently, $s_{d+1} = s_{d-1} - q_d s_d$ and $t_{d+1} = t_{d-1} - q_d t_d$. Since $\deg q_d = 1$, this takes another $O(k)$ operations.

https://doi.org/10.1017/CBO9781139856065.014 Published online by Cambridge University Press

The degrees of s_{d+1} and t_{d+1}, in the bottom row of $Q_d R$, are at most $n_0 - n_d = \lceil k/2 \rceil$. The entries of S have degrees $n_{d+1} - n_{k-1}, n_d - n_{k-1}, n_{d+1} - n_k, n_d - n_k$, all at most $d^* \le k/2$. Thus the computation of $S \cdot Q_d R$ takes at most another $8\mathsf{M}(k/2) + O(k)$, or $4\mathsf{M}(k) + O(k)$ operations.

Putting things together, we have a cost of at most $12\mathsf{M}(k) + O(k)$. Thus T satisfies the recursive inequalities

$$T(0) = 0, \quad T(k) \le 2T(\lfloor k/2 \rfloor) + 12\mathsf{M}(k) + ck \text{ for } k > 0,$$

for some constant $c \in \mathbb{R}$, and hence $T(k)$ is at most $(12\mathsf{M}(k) + O(k)) \log k$, by Lemma 8.2. □

We now state a more sophisticated version of Algorithm 11.4, which differs from it in the following respects.

- Some optimization is possible in step 4. First, since the remainders are going to be truncated anyway in step 7, it is sufficient to compute only the top $2(d^* + 2)$ coefficients of r_{d-1} and r_d, which we obtain by applying R to the vector with the truncated entries $r_0 \restriction 2k$ and $r_1 \restriction 2k - 1$. This will also allow us to drop the restriction $k \ge n/2$ for the cost estimate in Theorem 11.5.

  In addition, we can make use of the fact that the top $d - 1$ coefficients in the matrix-vector product vanish, and truncate at the top as well. This reduces the cost for step 4 to $6\mathsf{M}(k) + O(k)$ and the constant in Theorem 11.5 from 12 to 10.
- Using the strategy discussed in the paragraph before Algorithm 11.4, the improved algorithm also works when the degree sequence is not normal. Instead of returning R_k, the algorithm returns $h = \eta(k)$, as defined in (6), and R_h. In particular, the restriction $\deg r_0 > \deg r_1$ can be relaxed to $\deg r_0 \ge \deg r_1$.
- In addition to the matrix R_h, the improved algorithm also returns the quotients $q_1, \ldots, q_h$.

ALGORITHM 11.6 Half gcd.
Input: $r_0, r_1 \in F[x]$, $n = n_0 = \deg r_0 \ge \deg r_1 = n_1$, and $k \in \mathbb{N}$ with $0 \le k \le n$.
Output: $h = \eta(k) \in \mathbb{N}$ as defined in (6), $q_1, \ldots, q_h \in F[x]$, and $R_h \in F[x]^{2\times 2}$ as in (2).

1. **if** $r_1 = 0$ or $k < n_0 - n_1$ **then return** 0, the empty sequence, and $\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$
   else if $k = 0 = n_0 - n_1$ **then return** 1, $\dfrac{\mathrm{lc}(r_0)}{\mathrm{lc}(r_1)}$, and $\begin{pmatrix} 0 & 1 \\ 1 & -\dfrac{\mathrm{lc}(r_0)}{\mathrm{lc}(r_1)} \end{pmatrix}$
2. $d \longleftarrow \lceil k/2 \rceil$

https://doi.org/10.1017/CBO9781139856065.014 Published online by Cambridge University Press

3. **call** the algorithm recursively with $r_0 \restriction (2d-2)$, $r_1 \restriction (2d-2-(n_0-n_1))$ and $d-1$ (instead of k) as inputs, returning $\eta(d-1)$, $q_1,\dots,q_{\eta(d-1)}$, and $R = Q_{\eta(d-1)}\cdots Q_1$

4. $j \longleftarrow \eta(d-1)+1, \quad \delta \longleftarrow \deg R_{2,2}$,
$\begin{pmatrix} \tilde{r}_{j-1} \\ \tilde{r}_j \end{pmatrix} \longleftarrow R \begin{pmatrix} r_0 \restriction 2k \\ r_1 \restriction 2k-(n_0-n_1) \end{pmatrix}, \quad \begin{pmatrix} \tilde{n}_{j-1} \\ \tilde{n}_j \end{pmatrix} \longleftarrow \begin{pmatrix} \deg \tilde{r}_{j-1} \\ \deg \tilde{r}_j \end{pmatrix}$

5. **if** $\tilde{r}_j = 0$ or $k < \delta + \tilde{n}_{j-1} - \tilde{n}_j$ **then return** $j-1$, $q_1,\dots,q_{j-1}$, and R

6. $q_j \longleftarrow \tilde{r}_{j-1}$ quo $\tilde{r}_j, \quad Q_j \longleftarrow \begin{pmatrix} 0 & 1 \\ 1 & -q_j \end{pmatrix}$
$\tilde{r}_{j+1} \longleftarrow \tilde{r}_{j-1}$ rem $\tilde{r}_j, \quad \tilde{n}_{j+1} \longleftarrow \deg \tilde{r}_{j+1}$

7. $d^* \longleftarrow k-\delta-(\tilde{n}_{j-1}-\tilde{n}_j)$

8. **call** the algorithm recursively with input $\tilde{r}_j \restriction 2d^*$, $\tilde{r}_{j+1} \restriction (2d^*-(\tilde{n}_j-\tilde{n}_{j+1}))$ and d^* (instead of k), returning $\eta(d^*)$, $q_{j+1},\dots,q_h$, and $S = Q_{\eta(d^*)+j}\cdots Q_{j+1}$

9. $h \longleftarrow \eta(d^*)+j$
return h, $q_1,\dots,q_h$, and SQ_jR

Note that in the case of a normal degree sequence, we have $\eta(d-1) = d-1$ in step 3, $j = d$ and $\delta = d-1$ in step 4, the condition in step 5 is never satisfied, $d^* = k-d$ in step 7, $\eta(d^*) = d^*$ in step 8, and $h = k$ in step 9, so that Algorithm 11.4 is indeed a special case of Algorithm 11.6.

Before proving the correctness of the latter algorithm, let us see how it works in the above example.

EXAMPLE 11.2 (continued). Let $r_0 = f + x^3$ and $r_1 = g + x^2$, where f and g in $\mathbb{F}_7[x]$ are as in Example 11.2, and $k = 5$. The quotients in the Euclidean Algorithm for r_0 and r_1 are

$$q_1 = x+1, \quad q_2 = 4x, \quad q_3 = 3x+6, \quad q_4 = 6x^3+5$$

of degrees $m_1 = 1$, $m_2 = 1$, $m_3 = 1$, and $m_4 = 3$, and the length is $\ell = 4$. The algorithm proceeds as follows.

2. $d = \lceil 5/2 \rceil = 3$.

3. The recursive call with input $r_0 \restriction (2\cdot 3-2) = r_0 \restriction 6 = x^4+5x^3+3x^2+5$, $r_1 \restriction (6-(8-7)) = x^3+4x^2+4x+2$ and $d-1 = 2$ yields $\eta(2) = 2$, q_1, q_2, and

$$R = Q_2Q_1 = \begin{pmatrix} 0 & 1 \\ 1 & 3x \end{pmatrix}\begin{pmatrix} 0 & 1 \\ 1 & 6x+6 \end{pmatrix} = \begin{pmatrix} 1 & 6x+6 \\ 3x & 4x^2+4x+1 \end{pmatrix} = \begin{pmatrix} s_2 & t_2 \\ s_3 & t_3 \end{pmatrix}.$$

https://doi.org/10.1017/CBO9781139856065.014 Published online by Cambridge University Press

4. $j \longleftarrow 3$, $\delta \longleftarrow 2$, $r_0 \restriction 10 = x^2 r_0$, $r_1 \restriction 9 = x^2 r_1$, $\begin{pmatrix} \tilde{r}_2 \\ \tilde{r}_3 \end{pmatrix} = R \begin{pmatrix} r_0 \restriction 10 \\ r_1 \restriction 9 \end{pmatrix} = \begin{pmatrix} 2x^8 + x^7 + 2x^6 + 6x^5 + 6x^4 + 5x^3 + 6x^2 \\ 3x^7 + 6x^6 + 5x^5 + 6x^4 + 5x^3 + 3x^2 \end{pmatrix}$, $\tilde{n}_2 \longleftarrow 8$, and $\tilde{n}_3 \longleftarrow 7$.

5. $\tilde{r}_3 \neq 0$ and $k = 5 \geq 3 = \delta + \tilde{n}_2 - \tilde{n}_3$, so the condition is not satisfied.

6. Here q_3, Q_3, $\tilde{r}_4 = 4x^4 + x^3 + 2x^2$, and $\tilde{n}_4 = 4$ are computed.

7. $d^* \longleftarrow 2$.

8. The recursive call with input $\tilde{r}_3 \restriction (2 \cdot 2) = 3x^4 + 6x^3 + 5x^2 + 6x + 5$, $\tilde{r}_4 \restriction (2 \cdot 2 - (7-4)) = 4x + 1$ and d^* returns $\eta(2) = 0$ and $S = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$.

9. $h = 3$, q_1, q_2, q_3, and the matrix

$$\begin{aligned} SQ_3R &= \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 4x+1 \end{pmatrix} \begin{pmatrix} 1 & 6x+6 \\ 3x & 4x^2+4x+1 \end{pmatrix} \\ &= \begin{pmatrix} 3x & 4x^2+4x+1 \\ 5x^2+3x+1 & 2x^3+6x^2 \end{pmatrix} = \begin{pmatrix} s_3 & t_3 \\ s_4 & t_4 \end{pmatrix} \end{aligned}$$

are returned, which is correct because

$$\sum_{1 \leq i \leq 3} m_i = 3 \leq 5 < 6 = \sum_{1 \leq i \leq 4} m_i. \diamond$$

THEOREM 11.7.

Algorithm 11.6 works correctly and uses no more than $(22\mathsf{M}(k) + O(k)) \log k$ *additions and multiplications plus* $k+1$ *inversions, in total* $O(\mathsf{M}(k) \log k)$ *operations in* F. *The bound on the number of additions and multiplications drops to* $(10\mathsf{M}(k) + O(k)) \log k$ *if the degree sequence is normal.*

PROOF. Let ℓ be the number of division steps in the Euclidean Algorithm for (r_0, r_1), $n_i = \deg r_i$ for $0 \leq i \leq \ell$, and $m_i = \deg q_i$ for $1 \leq i \leq \ell$ as usual. If $r_1 = 0$ or $k < n_0 - n_1$, then $\eta(k) = 0$, and the algorithm correctly returns the identity matrix in step 1 (note that the empty product is defined to be the multiplicative neutral element). If $k = 0 = n_0 - n_1$, then the first quotient is $q_1 = \mathrm{lc}(r_0)/\mathrm{lc}(r_1)$, and the correct result is returned as well. Otherwise, $k \geq 1$, and we see by induction on k and Lemma 11.3 that the results of the recursive call in step 3 are correct. In particular, $\delta = m_1 + \cdots + m_{j-1} \leq d - 1 < k$ by (6) and the definition of d.

In step 4, we have that (r_{j-1}, r_j) and $(\tilde{r}_{j-1}, \tilde{r}_j)$ coincide up to $2(k - \delta)$, unless $\tilde{r}_j = 0$ or $k - \delta < \tilde{n}_{j-1} - \tilde{n}_j$, by arguments similar to the ones used in the proof of Lemma 11.1. In particular, $n_0 - n_{j-1} = 2k - \tilde{n}_{j-1}$ and $\tilde{n}_{j-1} - \tilde{n}_j = m_j = n_{j-1} - n_j$

https://doi.org/10.1017/CBO9781139856065.014 Published online by Cambridge University Press

if $\tilde{r}_j \neq 0$. If now $\tilde{r}_j = 0$ or $k < \delta + (\tilde{n}_{j-1} - \tilde{n}_j) = \delta + m_j$, then $\eta(k) = j-1$ and the algorithm returns the correct result in step 5. Otherwise, by Lemma 11.1, the correct quotient q_j is computed in step 6, and either $\tilde{r}_{j+1} = 0$ or $k - \delta - (\tilde{n}_{j-1} - \tilde{n}_j) < \tilde{n}_j - \tilde{n}_{j+1}$ or (r_j, r_{j+1}) and $(\tilde{r}_j, \tilde{r}_{j+1})$ coincide up to $2(k - \delta - (n_{j-1} - n_j)) = 2d^*$.

Again by induction and Lemma 11.3, the results of the recursive call in step 8 are correct, and

$$n_j - n_h = \sum_{j+1 \leq i \leq h} m_i \leq d^* < \sum_{j+1 \leq i \leq h+1} m_i = n_j - n_{h+1} \tag{7}$$

or else $h = \ell$. It remains to show that $h = \eta(k)$. By (7) we have

$$\begin{aligned} n_0 - n_h &= (n_0 - n_j) + (n_j - n_h) \leq (n_0 - n_j) + d^* \\ &< (n_0 - n_j) + (n_j - n_{h+1}) = n_0 - n_{h+1} \end{aligned}$$

or else $h = \ell$. But this implies $h = \eta(n_0 - n_j + d^*) = \eta(k)$ by (6) and the definition of d^*, and hence the final results in step 9 are correct.

As in the proof of Theorem 11.5, we may arrange things in such a way that the number of inversions in F performed during the recursive process is at most $k+1$.

Let $T(k)$ denote the number of additions and multiplications that the algorithm uses on input k. Steps 3 and 8 take $T(d-1)$ and $T(d^*)$ operations, respectively, for solving a subproblem of the same kind, together at most $2T(\lfloor k/2 \rfloor)$ since $d - 1 \leq \lfloor k/2 \rfloor$ and $d^* = k - (n_0 - n_j) < k - (d-1) = \lfloor k/2 \rfloor + 1$, by (6). We now analyze the cost for the polynomial multiplications, divisions, and additions in steps 1, 4, 6 and 9. The cost for step 1 is $O(1)$.

In step 4, the entries of R are $s_{j-1}, t_{j-1}, s_j, t_j$, by Lemma 3.8 (ii), and their degrees are $n_1 - n_{j-2}, n_0 - n_{j-2}, n_1 - n_{j-1}, n_0 - n_{j-1}$, by Lemma 3.10. All four degrees are at most $n_0 - n_{j-1} = \delta \leq d - 1 < k/2$. We have four multiplications of polynomials of degree at most $k/2$ by polynomials of degree at most $2k$, plus some additions, taking $8\mathsf{M}(k) + O(k)$ operations in F, as in the proof of Theorem 11.5. (As noted earlier, in the normal case, the bound drops to $6\mathsf{M}(k) + O(k)$.)

In step 6, computing the quotient of degree $m_j = \tilde{n}_{j-1} - \tilde{n}_j \leq k$ of the division of $\tilde{r}_{j-1}$ by $\tilde{r}_j$, of degree $\tilde{n}_j = 2k - (n_0 - n_j) = k + d^*$, takes $4\mathsf{M}(k) + O(k)$ operations, by Theorem 9.6. By partitioning the divisor $\tilde{r}_j$ into two blocks of sizes at most $k/2$, as in Exercise 8.35, the remainder can be computed using at most another $2\mathsf{M}(k) + O(k)$ operations, together no more than $6\mathsf{M}(k) + O(k)$.

In step 9, we first compute $Q_j R$, or equivalently, $s_{j+1} = s_{j-1} - q_j s_j$ and $t_{j+1} = t_{j-1} - q_j t_j$. This amounts to two multiplications of the quotient q_j of degree $n_{j-1} - n_j \leq k$ by a polynomial of degree at most $n_0 - n_{j-1} < k/2$ plus some additions, taking at most $2\mathsf{M}(k) + O(k)$.

The degrees of s_{j+1} and t_{j+1}, the lower row of $Q_j R$, are at most $n_0 - n_j \leq k$. The entries of S have degrees $n_{j+1} - n_{h-1}, n_j - n_{h-1}, n_{j+1} - n_h, n_j - n_h$, respectively,

https://doi.org/10.1017/CBO9781139856065.014 Published online by Cambridge University Press

all at most $n_j - n_h \le d^* \le k/2$. Thus the computation of $S \cdot Q_j R$ takes at most another $4\mathsf{M}(k/2) + 4\mathsf{M}(k) + O(k)$ or $6\mathsf{M}(k) + O(k)$ operations. In the normal case, we have $n_0 - n_j = \lceil k/2 \rceil + 1$, and the cost is only $4\mathsf{M}(k) + O(k)$.

Putting things together, we have a cost of at most $22\mathsf{M}(k) + O(k)$. Thus T satisfies the recursive inequalities

$$T(0) = 0, \quad T(k) \le 2T(\lfloor k/2 \rfloor) + 22\mathsf{M}(k) + ck \text{ for } k > 0,$$

for some constant $c \in \mathbb{R}$, and hence $T(k)$ is at most $(22\mathsf{M}(k) + O(k)) \log k$, by Lemma 8.2. In the normal case, the bound drops to $(10\mathsf{M}(k) + O(k)) \log k$. □

We have not attempted to determine the smallest possible constant in the place of 22. In fact, it is possible to prove a slightly better bound on the arithmetic cost for the "non-normal" case where not all the quotients have small degree. Strassen (1983) showed that in the nonscalar model of computation, where additions and multiplications by scalars are not counted and an interpolation algorithm proves (over an infinite field) that $\mathsf{M}(n) \in O(n)$, the cost for Algorithm 11.6 when $k = n$ can be bounded by $O(n \cdot H(m_1/m, \ldots, m_\ell/m))$, where $m = \sum_{1 \le i \le \ell} m_i$ and H is the entropy function, as in Section 10.1. This coincides with the bound of Theorem 11.7 in the normal case, since then $m_i/m = 1/n$ for $1 \le i \le \ell = n$.

Strassen also showed that the computation of all quotients of the Euclidean Algorithm in the nonscalar model requires at least about $n \cdot H(m_1/m, \ldots, m_\ell/m)$ field operations for almost all pairs of polynomials with quotient degrees $m_1, \ldots, m_\ell$. This lower bound shows that Algorithm 11.6 is *uniformly optimal* in the nonscalar model.

The q_i as calculated in step 5 of Algorithm 11.6 are exactly the quotients in the Euclidean Algorithm. Thus, taking $\mathsf{M}(n) \in O(n \log n \operatorname{loglog} n)$, Theorem 11.7 for $k = n$ implies that all the quotients q_i in the Euclidean Algorithm can be computed in time $O^\sim(n)$. Can we also compute all the remainders r_i together in softly linear time? The answer is: no. In the normal case, where always $\deg r_i = n - i$, the number of coefficients of $r_0, \ldots, r_\ell$ is

$$\sum_{0 \le i \le \ell} (\deg r_i + 1) = \sum_{0 \le i \le n} (n - i + 1) = (n^2 + 3n + 2)/2.$$

But this means that the output size is quadratic in the input size $2n$, and hence any algorithm that computes $r_0, \ldots, r_\ell$ requires at least $n^2/2$ field operations, since in some examples all output values are different (if the field is large enough), and therefore each requires at least one operation.

Algorithm 11.6 computes the product $R_{\eta(k)}$ of the Q-matrices in the Euclidean Algorithm such that the sum of the degrees of the corresponding quotients is roughly k. Given $f, g \in F[x]$, it is easy to compute the gcd of f and g and the

https://doi.org/10.1017/CBO9781139856065.014 Published online by Cambridge University Press

Bézout coefficients s,t from this matrix, by letting $k = n$. Then $\eta(k) = \ell$, s_ℓ and t_ℓ constitute the first row of the matrix R_ℓ, and

$$r_\ell = s_\ell f + t_\ell g.$$

Dividing by $\operatorname{lc}(r_\ell)$ gives $\gcd(f,g) = r_\ell/\operatorname{lc}(r_\ell)$ and the Bézout coefficients $s = s_\ell/\operatorname{lc}(r_\ell)$ and $t = t_\ell/\operatorname{lc}(r_\ell)$. Moreover, it is possible (as it is with the classical Extended Euclidean Algorithm) to compute any *single* row r_h, s_h, t_h for some $h \in \{1,\ldots,\ell\}$ when h is specified by a lower bound on $\deg r_h$, or equivalently, an upper bound on $\sum_{1\le i\le h} \deg q_i$, so that a number $k \in \mathbb{N}$ is given and h is determined as $h = \eta(k)$. This has applications in computer algebra and coding theory beyond the sole computation of gcds. Examples are: rational function reconstruction (Section 5.7), Padé approximation (Section 5.9), Cauchy interpolation (Section 5.8), decoding of BCH codes (the Berlekamp–Massey algorithm in Chapter 7), and fast sparse linear system solving (Chapter 12).

For completeness, we will now state such an algorithm returning all quotients as well as a complete row in the traditional Extended Euclidean Algorithm.

ALGORITHM 11.8 Fast Extended Euclidean Algorithm.
Input: $f,g \in F[x] \setminus \{0\}$ with $n = \deg f \ge \deg g = m$, and $k \in \mathbb{N}$ with $0 \le k \le n$.
Output: The quotients $q_1,\ldots,q_h \in F[x]$ and polynomials s_h,t_h as in (1) for $r_0 = f$ and $r_1 = g$, where $h = \eta(k)$ is defined as in (6), and the remainder r_h.

1. **call** Algorithm 11.6 with input f, g, and k, to compute $h = \eta(k)$, $q_1,\ldots,q_h$, and $R_h = \begin{pmatrix} s_h & t_h \\ s_{h+1} & r_{h+1} \end{pmatrix}$
2. **return** $q_1,\ldots,q_h$, s_h,t_h, and $s_h f + t_h g$

By Theorem 11.7, the cost for computing only the quotients and s_h,t_h in this algorithm is $O(\mathsf{M}(k)\log k)$ additions and multiplications in F plus at most $k+1$ inversions. The additional cost for computing r_h is $O(\mathsf{M}(n))$ additions and multiplications. Note that it is easy to compute the corresponding row in the monic Extended Euclidean Algorithm by multiplying r_h, s_h, t_h by $\operatorname{lc}(r_h)^{-1}$, at a cost of another $O(n)$ field operations.

COROLLARY 11.9.
For polynomials $f,g \in F[x]$ of degree at most n, all of the following can be computed with $O(\mathsf{M}(n)\log n)$ additions and multiplications plus at most $n+2$ inversions, or $O^\sim(n)$ operations in F:

- $r_\ell = \gcd(f,g)$,
- *the Bézout coefficients $s,t \in F[x]$ with $sf + tg = r_\ell$,*

https://doi.org/10.1017/CBO9781139856065.014 Published online by Cambridge University Press

- *the entries $r_h, s_h, t_h \in F[x]$ of an arbitrary row in the (traditional or monic) Extended Euclidean Algorithm for f, g,*
- *the quotients $q_1, \ldots, q_\ell$ in the traditional Euclidean Algorithm for f, g.*

We give explicit constants for the first two statements of the above corollary.

THEOREM 11.10.
Let $f, g \in F[x] \setminus \{0\}$ with $\deg g \le \deg f \le n$.

(i) With at most $(22\mathsf{M}(n) + O(n)) \log n$ additions and multiplications plus $n+2$ inversions in F, we can decide whether f and g are coprime, and if so, compute the Bézout coefficients $s, t \in F[x]$ such that $sf + tg = 1$.

(ii) If the degree sequence is normal, then we can compute the monic $\gcd(f, g)$ and the Bézout coefficients $s, t \in F[x]$ such that $sf + tg = \gcd(f, g)$ using at most $10\mathsf{M}(n) \log n + 2\mathsf{M}(n) + O(n \log n)$ additions and multiplications plus $n+2$ inversions.

PROOF. (i) We see from Lemma 3.10 that the gcd is constant if and only if $\deg s_{\ell+1} = \deg g$ and $\deg t_{\ell+1} = \deg f$. If so, then we return $s = s_\ell / r_\ell$ and $t = t_\ell / r_\ell$. The cost for computing the constant polynomial $r_\ell = s_\ell(0) f(0) + t_\ell(0) g(0)$ and dividing s_ℓ, t_ℓ by it is one inversion and $O(n)$ multiplications and additions, and the claim follows from Theorem 11.7 with $k = n$.

(ii) The degrees of s_ℓ and t_ℓ are less than n, by Lemma 3.10, and hence computing $r_\ell = s_\ell r_0 + t_\ell r_1$ amounts to $2\mathsf{M}(n) + O(n)$ arithmetic operations. □

COROLLARY 11.11.
Let $f \in F[x]$ of degree $n > 0$. A product in the residue class ring $F[x]/\langle f \rangle$ can be computed with $6\mathsf{M}(n) + O(n)$ arithmetic operations in F, and an inverse with no more than $(22\mathsf{M}(n) + O(n)) \log n$ operations. Thus one arithmetic operation in $F[x]/\langle f \rangle$ takes $O^\sim(n)$ arithmetic operations in F.

Applying Corollaries 10.12, 10.17, 11.9, and 11.11 to the analysis of the modular Extended Euclidean Algorithms 6.36 and 6.59, we obtain the following result.

COROLLARY 11.12.
Let F be a field with at least $(6n+3)d$ elements and $f, g \in F[y][x]$ of degree at most n in x and at most d in y.

(i) With an expected number of $O(d\,\mathsf{M}(n) \log n + n\,\mathsf{M}(d) \log d)$ or $O^\sim(nd)$ arithmetic operations in F, we can compute the gcd of f and g.

(ii) A single row of the EEA for f, g can be computed with $O(n\,\mathsf{M}(nd) \log(nd))$ or $O^\sim(n^2 d)$ operations in F.

https://doi.org/10.1017/CBO9781139856065.014 Published online by Cambridge University Press

The Euclidean Algorithm for integers. The method also works for integers, although there are some complications due to the carries. But Corollary 11.9 is true also for integers when the cost measure is the number of word operations instead of field operations. Wang & Pan (2003) describe a suitable algorithm; see also Pan & Wang (2004). The output row i is specified by an integer h and the condition $s_i \le 2^h < s_{i+1}$. For inputs of at most n words, it uses $O(\mathsf{M}(n)\log h)$ word operations.

COROLLARY 11.13.
For an integer $m \in \mathbb{N}$ of length n, one arithmetic operation in the residue class ring $\mathbb{Z}_m$ can be performed using $O^{\sim}(n)$ word operations. More precisely, the cost is $O(n)$ for an addition, $O(\mathsf{M}(n))$ for a multiplication, and $O(\mathsf{M}(n)\log n)$ for an inversion or a division.

The following is the analog of Corollary 11.12 for integer polynomials.

COROLLARY 11.14.
Let $f, g \in \mathbb{Z}[x]$ of degree at most n and with max-norm at most A.

(i) The gcd *of f and g can be computed using an expected number of*

$$O\Big(\mathsf{M}(n)\log n\cdot(n+\log A)\cdot\mathsf{M}(\log(n\log(nA)))\cdot\operatorname{loglog}(n\log(nA)) \\ +n\,\mathsf{M}((n+\log A)\log(n\log(nA)))\cdot(\log n+\operatorname{loglog}A)\Big)$$

or $O^{\sim}(n^2+n\log A)$ word operations.

(ii) A single row in the EEA of f, g can be computed taking

$$O(\mathsf{M}(n)\log n\cdot\mathsf{M}(n\log(nA))\cdot\log(n\log(nA))) \text{ or } O^{\sim}(n^2\log A)$$

word operations.

11.2. Subresultants via Euclid's algorithm

We have seen in Section 6.10 how to express each result of the Extended Euclidean Algorithm for polynomials via subresultants. Now we turn this question around: can we express subresultants, in particular the resultant, via the results of the Euclidean Algorithm? The answer is yes.

This section may be skipped at first reading. The main application of its result, the so-called *fundamental theorem on subresultants*, is to make the fast Euclidean Algorithm of this chapter also work for resultants, which appear in Section 6.8 and Chapters 15, 22, and 23.

https://doi.org/10.1017/CBO9781139856065.014 Published online by Cambridge University Press

We start by relating the two subresultants corresponding to one division step. We write $S_\kappa(f,g)$ for the κth submatrix of $\mathrm{Syl}(f,g)$, as in Section 6.10, and $\sigma_\kappa(f,g) = \det S_\kappa(f,g)$.

LEMMA 11.15. *Let $f,g,r \in F[x]$ be nonzero polynomials with degrees $n \geq m > d$, respectively, $r = f \bmod g$, and $0 \leq \kappa \leq d$. Then*

$$\sigma_\kappa(f,g) = (-1)^{(n-\kappa)(m-\kappa)} \operatorname{lc}(g)^{n-d} \sigma_\kappa(g,r).$$

PROOF. There exists $q \in F[x]$ of degree $n-m$ such that $f = qg + r$. Denoting by f_j, g_j, q_j, r_j the coefficients of f,g,q,r, respectively, we can rewrite this as

$$\begin{pmatrix} f_n \\ f_{n-1} \\ \vdots \\ \vdots \\ \vdots \\ \vdots \\ \vdots \\ f_0 \end{pmatrix} - \underbrace{\begin{pmatrix} g_m & & & \\ g_{m-1} & g_m & & \\ \vdots & & \ddots & \\ \vdots & & & g_m \\ g_0 & & & \vdots \\ & \ddots & & \vdots \\ & & \ddots & \vdots \\ & & & g_0 \end{pmatrix}}_{n-m+1} \begin{pmatrix} q_{n-m} \\ \vdots \\ \vdots \\ q_0 \end{pmatrix} = \begin{pmatrix} 0 \\ \vdots \\ \vdots \\ \vdots \\ 0 \\ r_d \\ \vdots \\ r_0 \end{pmatrix}. \tag{8}$$

The first column of $S_\kappa(f,g) \in F^{(n+m-2\kappa)\times(n+m-2\kappa)}$, defined on page 180, is

$$\begin{pmatrix} f_n \\ f_{n-1} \\ \vdots \\ f_{2\kappa-m+1} \end{pmatrix}, \tag{9}$$

and the second summand on the left hand side of (8), extended by zeroes or truncated—as necessary—to length $n+m-2\kappa$, is a linear combination of the columns in the right part of $S_\kappa(f,g)$. Thus the column

$$(0,\ldots,0,r_d,\ldots,r_{2\kappa-m+1})^T,$$

where ^{T} denotes transposition, is a linear combination of columns of $S_\kappa(f,g)$, with coefficient 1 at the column (9), and we may substitute it for the column (9) in $S_\kappa(f,g)$ without changing the value of the determinant; as usual, coefficients out of range are set to zero.

https://doi.org/10.1017/CBO9781139856065.014 Published online by Cambridge University Press

Similarly, we may replace each other column of f_j's in $S_\kappa(f,g)$ by the corresponding column of r_j's. Next, we move the columns of r_j's in order to the right side of the matrix. This changes the sign of the determinant $(n-\kappa)(m-\kappa)$ times.

Together we find

$$\sigma_\kappa(f,g) = \det S_\kappa(f,g)$$

$$= (-1)^{(n-\kappa)(m-\kappa)} \det \begin{pmatrix} g_m & & & & & & \\ g_{m-1} & \ddots & & r_d & & & \\ \vdots & & g_m & \vdots & \ddots & & \\ \vdots & & \vdots & \vdots & & r_d & \\ \vdots & & \vdots & \vdots & & \vdots & \\ g_{2\kappa-n+1} & \cdots & g_\kappa & r_{2\kappa-m+1} & \cdots & r_\kappa \end{pmatrix}.$$

$$\underbrace{\qquad\qquad}_{n-\kappa} \qquad \underbrace{\qquad\qquad}_{m-\kappa}$$

This matrix is of block form

$$\begin{pmatrix} D & 0 \\ * & S_\kappa(g,r) \end{pmatrix},$$

$$\underbrace{}_{n-d} \quad \underbrace{}_{m+d-2\kappa}$$

where D is a lower triangular $(n-d)\times(n-d)$ matrix with diagonal entries equal to g_m. Thus

$$\sigma_\kappa(f,g) = (-1)^{(n-\kappa)(m-\kappa)} g_m^{n-d} \sigma_\kappa(g,r). \ \square$$

We recall the degree sequence $(n_0,\dots,n_\ell)$, with $n_i = \deg r_i$ for all i. Using the above lemma repeatedly, we arrive at the following celebrated theorem.

THEOREM 11.16 *Fundamental Theorem on Subresultants.* *Let $f = r_0, g = r_1 \in F[x]\setminus\{0\}$ of degree $n \geq m$, respectively, and $n_i = \deg r_i$ and $\alpha_i = \mathrm{lc}(r_i)$ the degrees and the leading coefficients from the traditional Euclidean Algorithm 3.5 for $0 \leq i \leq \ell$.*

(i) For $0 \leq \kappa \leq m$, the κth subresultant of (f,g) is

$$\sigma_\kappa = \det S_\kappa = \begin{cases} (-1)^{\tau_i} \alpha_i^{n_{i-1}-n_i} \displaystyle\prod_{1\leq j<i} \alpha_j^{n_{j-1}-n_{j+1}} & \text{if } \kappa = n_i \text{ for some } i \leq \ell, \\ 0 & \text{otherwise}, \end{cases}$$

where $\tau_i = \sum_{1\leq j<i}(n_{j-1}-n_i)(n_j-n_i)$.

https://doi.org/10.1017/CBO9781139856065.014 Published online by Cambridge University Press

(ii) The subresultants satisfy for $1 \le i < \ell$ the recursive formulas

$$\sigma_m = \alpha_1^{n-m}, \quad \sigma_{n_{i+1}} = (-1)^{(n_i-n_{i+1})(n-n_{i+1}+i+1)}(\alpha_i\alpha_{i+1})^{n_i-n_{i+1}}\sigma_{n_i}.$$

PROOF. (i) We know from Theorem 6.48 that σ_κ vanishes in the second case. So we may assume that $\kappa = n_i$ for some $i \le \ell$. This i is unique, so that the expressions in the claim are well defined.

Induction on h for $0 \le h < i$, using Lemma 11.15, shows that

$$\sigma_\kappa(r_0, r_1) = \sigma_\kappa(r_h, r_{h+1}) \prod_{1 \le j \le h} (-1)^{(n_{j-1}-\kappa)(n_j-\kappa)} \alpha_j^{n_{j-1}-n_{j+1}}.$$

The claim follows from the case $\kappa = n_i$ and $h = i-1$, together with $\sigma_{n_i}(r_{i-1}, r_i) = \alpha_i^{n_{i-1}-n_i}$.

(ii) follows from (i) by calculating $\tau_{i+1} - \tau_i$ modulo 2. □

Exercise 11.10 shows that $(-1)^{\tau_i} = 1$ when the degree sequence is normal.

We illustrate the theorem with an example.

EXAMPLE 11.17. Let $f = r_0 = x^3 + 2x^2 + 3x + 4$ and $g = r_1 = 3x^2 + 2x + 1$ in $\mathbb{Q}[x]$. Then the Euclidean Algorithm computes

$$\begin{aligned} r_0 &= q_1 r_1 + r_2 = \left(\frac{1}{3}x + \frac{4}{9}\right) r_1 + \frac{16}{9}x + \frac{32}{9}, \\ r_1 &= q_2 r_2 + r_3 = \left(\frac{7}{16}x - \frac{9}{4}\right) r_2 + 9, \\ r_2 &= q_3 r_3 = \left(\frac{16}{27}x + \frac{32}{27}\right) r_3. \end{aligned}$$

Thus $\alpha_0 = 1, \alpha_1 = 3, \alpha_2 = 16/9, \alpha_3 = 9$. The subresultants of (f, g) are

$$\begin{aligned} \sigma_2 &= \sigma_{n_1} = \det(3) = 3 = \alpha_1, \\ \sigma_1 &= \sigma_{n_2} = \det \begin{pmatrix} 1 & 3 & 0 \\ 2 & 2 & 3 \\ 3 & 1 & 2 \end{pmatrix} = 16 = (-1)^{\tau_2}\alpha_2\alpha_1^2, \\ \sigma_0 &= \sigma_{n_3} = \det \begin{pmatrix} 1 & 0 & 3 & 0 & 0 \\ 2 & 1 & 2 & 3 & 0 \\ 3 & 2 & 1 & 2 & 3 \\ 4 & 3 & 0 & 1 & 2 \\ 0 & 4 & 0 & 0 & 1 \end{pmatrix} = 256 = (-1)^{\tau_3}\alpha_3\alpha_2^2\alpha_1^2. \diamond \end{aligned}$$

https://doi.org/10.1017/CBO9781139856065.014 Published online by Cambridge University Press

Theorem 11.7 implies the following.

COROLLARY 11.18.
Let $f,g \in F[x] \setminus \{0\}$ have degrees $n = n_0 \geq n_1$, and let $0 \leq k \leq n$. Then all subresultants σ_j for $n_1 \geq j \geq (n-k)$ of (f,g) can be calculated with at most $(22\mathsf{M}(k) + O(k)) \log k$ operations in F.

PROOF. We use Algorithm 11.8 with input f,g and k to compute the quotients $q_1,\ldots,q_h$, where $h = \eta(k)$. By Theorem 6.53 (b), we can compute the $\alpha_i = \alpha_{i-1}/\operatorname{lc}(q_i)$ from α_0 and the leading coefficients of the quotients q_i using at most k multiplications and inversions each. Then we compute the subresultants up to σ_{n-k} along the recursion formula (ii) from Theorem 11.16. This takes at most $2k$ additional multiplications, and Theorem 11.7 implies the claim. □

Because of its importance, we highlight the case of the resultant, which corresponds to $\kappa = 0$ in Theorem 11.16 and to $k = n$ in Corollary 11.18.

COROLLARY 11.19.
Let $f,g \in F[x] \setminus \{0\}$, and let $n = n_0 \geq n_1 \geq \cdots \geq n_\ell$ and $\alpha_0,\ldots,\alpha_\ell$ be the degrees and the leading coefficients, respectively, of the remainders in the Euclidean Algorithm for (f,g).

If $\deg \gcd(f,g) = n_\ell \geq 1$, then $\operatorname{res}(f,g) = 0$. Otherwise, if $n_\ell = 0$,

$$\operatorname{res}(f,g) = (-1)^\tau \alpha_\ell^{n_{\ell-1}} \prod_{1 \leq j < \ell} \alpha_j^{n_{j-1}-n_{j+1}},$$

where $\tau = \sum_{1 \leq j < \ell} n_{j-1} n_j$. This resultant can be calculated using no more than $(22\mathsf{M}(n) + O(n)) \log n$ operations in F.

Corollary 11.18 implies that in fact all subresultants can be computed within the same time bound. This can be used to replace the rational number reconstruction (or Cauchy interpolation) in the modular EEA algorithms of Section 6.11, as follows. We multiply the modular images of the ith row r_i, s_i, t_i of the EEA with the modular image of the subresultant σ_{n_i}, for all "lucky" primes, and reconstruct $\sigma_{n_i} r_i, \sigma_{n_i} s_i, \sigma_{n_i} t_i$, which have integral (or polynomial) coefficients, by the fast Chinese Remainder Algorithm (or fast interpolation), for all i.

COROLLARY 11.20.
We can compute all subresultants of two polynomials $f,g \in \mathbb{Z}[x]$ of degree at most n and with max-norm at most A using $O(\mathsf{M}(n) \log n \cdot \mathsf{M}(n \log(nA)) \log(n \log(nA)))$ or $O^\sim(n^2 \log A)$ word operations.

https://doi.org/10.1017/CBO9781139856065.014 Published online by Cambridge University Press

PROOF. We modify the small primes modular EEA of Section 6.11 so that in addition it computes all subresultants modulo each small prime, using the leading coefficients of the quotients from the fast Euclidean Algorithm, as in the proof of Corollary 11.18, and recover them from their modular images by Chinese remaindering. By Corollary 11.18, the additional cost is negligible, and Corollary 11.14 implies the claim. □

The proof of the following result for bivariate polynomials is analogous.

COROLLARY 11.21.
We can compute all subresultants of two polynomials $f, g \in F[y][x]$ of degree at most n in x and at most d in y, where F is a field, using $O(n\mathsf{M}(nd)\log(nd))$ or $O^{\sim}(n^2 d)$ arithmetic operations in F.

Since all subresultants together have about $n^2 d$ coefficients, at least in a generic sense, the above result is—up to logarithmic factors—optimal. However, this is no longer true if we are only interested in a particular subresultant, for example, the resultant, and it is not clear to us whether an $O^{\sim}(nd)$ algorithm exists. Similar remarks apply to the integer case.

Notes. 11.1. The idea for a fast gcd algorithm is due to Lehmer (1938); later work includes Knuth (1970), Schönhage (1971), Moenck (1973), Aho, Hopcroft & Ullman (1974), §8.9, Schwartz (1980), Brent, Gustavson & Yun (1980), and Strassen (1983); the first three papers actually deal with integers. Brent, Gustavson & Yun apply their algorithm to computing Padé approximants and solving Toeplitz (or Hankel) systems of linear equations, and show that the two problems are equivalent. They also note that the "HGCD" algorithm from Aho, Hopcroft & Ullman does not always return the correct result in non-normal cases where not all quotients have degree 1. The Fast Extended Euclidean Algorithm 11.4 in the 1999 and 2003 editions of this text also contained an error. The concept of and notation for "coinciding" is from Strassen's paper. Pan (1997) gives a (sometimes) faster computation for Padé approximants and the decoding of BCH codes.
11.2. Lemma 11.15 for the resultant is in Gordan (1885), §145, Haskell (1891/92), and "well-known" to Swan (1962). The last paper also contains implicitly Theorem 11.16. In the traditional, and in Sturm's, variant of the Euclidean Algorithm (Exercise 4.32), the (sub)resultants of consecutive entries are all identical, up to sign. The fundamental theorem on subresultants is in Collins (1967) and Brown & Traub (1971); for our development, it takes a back seat compared to the results of Chapter 6. Exercise 11.8 provides a simple proof of the special case $\kappa = 0$ of Lemma 11.15.

Lickteig & Roy (1996, 2001) and Reischert (1997) give non-modular methods for computing resultants of polynomials in $\mathbb{Z}[x]$ or $F[y,x]$ and prove running time estimates which are—up to logarithmic factors—within the bounds from Corollaries 11.20 and 11.21.

Exercises.

11.1 Prove that for a ring R and each $k \in \mathbb{N}$, "coinciding up to k" is an equivalence relation on $R[x] \setminus \{0\} \times R$.

https://doi.org/10.1017/CBO9781139856065.014 Published online by Cambridge University Press

11.2 Let F be a field supporting the FFT. In the text, we have given fast $O(n\log n)$ and $O(n\log^2 n)$ algorithms, respectively, for the following six problems for polynomials in one variable of degree less than n over F: multiplication, division with remainder, inverse modulo x^n, evaluation at n points, interpolation at n points, and greatest common divisor. For each of these problems, describe in one sentence of at most 20 words how the algorithm for it depends on the FFT and the algorithms for the other problems, and what method is used for the "dependency".

11.3 Let F be a field, $k \in \mathbb{N}$, and $f, g, f^*, g^* \in F[x]$ nonzero. Prove or disprove the following "converse" of Lemma 11.1: If $g \upharpoonright k = g^* \upharpoonright k$ and f quo $g = f^*$ quo g^* has degree k, then $f \upharpoonright 2k = f^* \upharpoonright 2k$.

11.4 Let F be a field, $f \in F[x]$ of degree n, and $m_1, \ldots, m_r \in F[x]$ such that $\sum_{1\le i\le r} \deg m_i \le n$. Prove that $\gcd(f, m_1), \ldots, \gcd(f, m_r)$ can be computed with $O(\mathsf{M}(n)\log n)$ operations in F. Hint: Corollary 10.17.

11.6 In this exercise, you are to show how the Half gcd algorithm 11.6 can be sped up if FFT multiplication is used. Suppose that F is a field that supports the FFT. We know that the entries of SQ_jR in step 9 are of degree at most $n_0 - n_h \le k$, and it is sufficient to compute all of them modulo $x^\kappa - 1$, where $\kappa \in \mathbb{N}$ is the least power of 2 strictly larger than k. Then the product matrix can be computed by evaluating all entries of S, Q_j, R at the primitive κth roots of unity, performing the matrix products pointwise, and interpolating to obtain the result. Count the number of κ-point FFTs computed, and compare your result to the number of κ-point FFTs that the usual approach of separately computing all polynomial products takes.

11.7* You are to modify Algorithm 11.4 (Half gcd for normal degree sequence) so that in addition the two remainders $\begin{pmatrix} r_k \\ r_{k+1} \end{pmatrix} = R_k \begin{pmatrix} r_0 \\ r_1 \end{pmatrix}$ are output. You may assume that $n_0 \le 2k$. Analyze your algorithm carefully. The goal is to obtain a cost of at most $(10\mathsf{M}(k) + O(k))\log k$ field operations. Hint: In your modified step 4, you only have to compute $R \begin{pmatrix} r_0^* \\ r_1^* \end{pmatrix}$, where r_0^*, r_1^* are the "lower parts" of r_0, r_1, in order to obtain r_{d-1} and r_d, and a similar computation in step 8 gives r_k, r_{k+1}. Trace your algorithm on the data of Example 11.2. Can you explain why this algorithm outputs more than Algorithm 11.4 and still uses fewer operations?

11.8 Give an alternative proof of Lemma 11.15 for $k = 0$ using Exercise 6.12.

11.9 Let F be a field and $n, e_1, \ldots, e_d \in \mathbb{N}$, with $e_1 > \cdots > e_d$ and $e_1 + \cdots + e_d \le n$.

(i) Let $f, g \in F[x]$ of degree at most n, and assume that $e_1, \ldots, e_d$ occur in the degree sequence of the EEA for f and g. Show that the remainders of degrees $e_1, \ldots, e_d$ can be computed with $O(\mathsf{M}(n)\log n)$ operations in F.

(ii) Let $f, g \in F[y][x]$ of degree at most n in x and at most d in y, and assume that $\#F \ge (6n+3)d$ and $e_1, \ldots, e_d$ occur in the degree sequence of the EEA for f and g in $F(y)[x]$. Show that the remainders of degrees $e_1, \ldots, e_d$ in x can be computed with $O(n\mathsf{M}(nd)\log(nd))$ operations in F.

11.10 Prove that $(-1)^{\tau_i} = 1$ for all i in Theorem 11.16 in case the degree sequence is normal.

Research problems.

11.11 Determine the smallest possible constant $c \le 22$ such that your favorite fast Euclidean Algorithm works in time $(c\mathsf{M}(k) + O(k))\log k$.

11.12 Can you find an algorithm for computing the resultant of two polynomials $f, g \in F[x, y]$ of degree at most n in x and at most d in y taking $O^\sim(nd)$ operations in the field F?

11.13 Implement carefully fast algorithms for large instances of the problems discussed in this book.

https://doi.org/10.1017/CBO9781139856065.014 Published online by Cambridge University Press

Общеизвестно, что задача обращения матриц [. . .]
является одной из центральных и трудных задач теории
матриц. [. . .] К сожалению, несмотря
на обширную литературу, посвященную этому вопросу,
проблема во многих её аспектах требует
дальнейшего углублённого исследования.[1]

Iosif Semenovich Iohvidov (1974)

For what is the theory of determinants? It is an algebra upon algebra;
a calculus which enables us to combine and foretell the results of
algebraical operations, in the same way as algebra itself enables us to
dispense with the performance of the special operations of arithmetic.
All analysis must ultimately clothe itself under this form.

James Joseph Sylvester (1851)

Lorsqu'il n'est pas en notre pouvoir de discerner les plus vraies
opinions, nous devons suivre les plus probables.[2]

René Descartes (1637)

[1] It is commonly known that the problem of the inversion of matrices [. . .] is one of the central and difficult problems in matrix theory. [. . .] Unfortunately, in spite of extensive literature dealing with this question, the problem requires in many of its aspects a further and profound investigation.

[2] When it is not in our power to tell the most correct opinions apart, we ought to follow the most probable ones.

https://doi.org/10.1017/CBO9781139856065.014 Published online by Cambridge University Press